Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ 1

з дисципліни «Хмарні обчислення»

на тему: Паралельне шифрування Цезаря та XOR у системі ПАРКС (PARCS-Python) у хмарному середовищі Google Cloud Platform

Студента 4 курсу
групи ТК-41
спеціальності 122 «Комп'ютерні науки»
Тимченка Олександра

Київ 2026

Зміст

Мета роботи **2**

Вступ **2**

1. Платформа ПАРКС та PARCS-Python **2**

2. Постановка задачі та ідея паралельного алгоритму **3**

3. Реалізація **4**

4. Розгортання системи ПАРКС у Google Cloud Platform **5**

5. Експерименти **7**

5.1. Режим local 8

5.2. Режим scatter 10

6. Аналіз результатів **11**

Висновки **12**

Література **13**

Додаток А. Файл розв'язку solver.py **14**

Додаток Б. Інструкція з розгортання та запуску **24**

Б.1. Підготовка Google Cloud 24

Б.2. Створення кластера 24

Б.3. Запуск експериментів 24

Б.4. Видалення ресурсів 25

Додаток В. Приклад вихідного файлу (local_w4.txt) **26**

Мета роботи

Обрати задачу паралельного програмування (шифрування та розшифрування тексту шифром Цезаря і XOR) та реалізувати її для платформи ПАРКС (PARCS-Python). Розгорнути систему ПАРКС у хмарному середовищі Google Cloud Platform, провести тестування послідовного та паралельного алгоритмів для різної кількості воркерів (master і 1--4 воркери), порівняти результати та оцінити прискорення й ефективність паралельної програми.

Вступ

Шифр Цезаря і XOR-шифрування належать до найпростіших симетричних шифрів. Шифр Цезаря зсуває кожен байт тексту на фіксовану величину: $c_i = (p_i + k) \bmod 256$. XOR-шифрування накладає на текст ключ, що періодично повторюється: $c_i = p_i \oplus key[i \bmod |key|]$. Обидві операції побайтово незалежні, тому текст можна розбити на неперервні шматки й обробити їх паралельно на різних вузлах. Саме така «ідеально паралельна» структура задачі робить її зручною для порівняння двох речей: наскільки добре масштабуються власне обчислення і скільки коштує передача даних між вузлами.

Робота є портом власної MPI-реалізації на C++ (`serial.cpp`, `parallel.cpp`) на платформу PARCS-Python: збережено ті самі алгоритми, той самий ключ XOR із глобальним зсувом позиції і той самий розподіл даних між процесами ($count_i = rest/(p - i)$). У звіті досліджено два паралельні режими: режим `local`, у якому дані вже лежать на вузлах, і режим `scatter`, дослівний порт MPI-версії, де майстер розсилає текст воркерам і збирає результат.

Повний код проєкту, вхідні файли, результати вимірювань, графіки та інструкція з розгортання розміщені в репозиторії GitHub: `https://github.com/altiebalti/parcs-caesar-xor`.

1. Платформа ПАРКС та PARCS-Python

Технологія ПАРКС (Паралельні Асинхронні Рекурсивно Керовані Системи) запропонована А. В. Анісімовим як універсальний засіб побудови систем паралельної обробки інформації [1]. Її основу складають дві ідеї. Перша: замість нової мови програмування користувачеві пропонується базова алгоритмічна мова (Java, C, Python тощо), розширена невеликим набором засобів паралелізму, тому отримують ПАРКС-Java, ПАРКС-C і так далі. Друга: поняття керуючого простору, тобто множини точок, у які розподіляються алгоритмічні модулі, з визначеними законами взаємодії між ними. Базисний алгоритмічний модуль є послідовним дискретним перетворювачем із керуючою та інформаційною частинами й засобами взаємодії та синхронізації з іншими модулями за принципом рандеву Хоара; зі складених модулів шляхом композицій будуються складніші. ПАРКС-засоби дозволяють динамічно створювати і знищувати паралельні процеси та перекомутовувати зв'язки між ними, поєднуючи рекурсію й паралельність [1, розділ 2].

PARCS-Python [2] є реалізацією цієї технології для мови Python. Система складається з майстра (master) і воркерів (workers), кожен вузол працює в Docker-контейнері `hummer12007/parcs-node` (Python 2.7, Flask для веб-інтерфейсу та Pyro4 для віддалених викликів). Майстер надає веб-інтерфейс на порту 8080, через який додають задачі: файл розв'язку та вхідний файл. Розв'язок є Python-модулем із класом `Solver`, який має конструктор `__init__(workers, input_file_name, output_file_name)` і метод `solve()`. На майстрі `workers` є списком проксі Pyro4 на воркери, на воркерах клас завантажується без аргументів. Методи, позначені декоратором `@expose`, стають доступними для віддаленого виклику; виклик повертає об'єкт `FutureResult`, значення якого читають через `.value`, що дає асинхронність: майстер може запустити обчислення на всіх воркерах і лише потім чекати результатів. Аргументи та результати серіалізуються (serpent), тому двійкові дані передаються у вигляді base64-рядків.

2. Постановка задачі та ідея паралельного алгоритму

Вхідними даними є обсяг тексту N байтів, зсув Цезаря $k = 3$, ключ XOR `SecretKey123!@#` (15 байтів) і зерно генератора. Текст генерується детерміновано блоками по 1 МБ, кожен блок зі свого зерна $(seed, idx)$, тому за будь-якої кількості воркерів дані однакові, а md5-відбитки шифротексту мають збігатися в усіх режимах. Для кожного запуску перевіряється, що розшифрований текст дорівнює вихідному (`PASSED`/`FAILED`).

Паралельний алгоритм ґрунтується на побайтовій незалежності обох шифрів: текст ділиться на p неперервних шматків, кожен воркер шифрує й розшифровує свій шматок. Єдина нетривіальна деталь стосується XOR: ключ накладається не з початку шматка, а з глобальної позиції байта в тексті, $c_i = p_i \oplus key[(offset + i) \bmod |key|]$, інакше результат залежав би від кількості воркерів.

Реалізовано три режими, які задаються рядком `mode=` у вхідному файлі:

- **serial**: послідовний алгоритм; увесь конвеєр (генерація, шифрування й розшифрування Цезарем та XOR, перевірка) виконує майстер, воркери не використовуються. Майстер є такою самою VM, як і воркери, тому порівняння чесне.
- **local**: основний паралельний режим. Майстер ділить блоки між воркерами порівну, кожен воркер сам генерує свою частину тексту (модель «дані вже лежать на вузлах»), шифрує і розшифровує її, перевіряє результат і повертає майстру лише контрольні суми й таймінги. По мережі передаються лише параметри, тому очікуване прискорення близьке до лінійного.
- **scatter**: дослівний порт MPI-версії. Майстер генерує весь текст, розсилає шматки воркерам (аналог `MPI_Scatterv`), збирає зашифроване (`MPI_Gatherv`), розсилає знову для розшифрування і збирає ще раз, окремо для Цезаря і для XOR. Як і в C++ версії, генерація й перевірка не входять у виміряний час. Цей режим показує, скільки коштує передача даних порівняно з самими обчисленнями.

Прискорення для режиму `local` обчислюється як $S = T_{serial}/T_{local}$ за повним часом

конвеєра; для `scatter` базою є сума часу Цезаря і XOR у послідовному режимі, так само, як у таблиці MPI-версії. Ефективність $E = S/p \cdot 100\%$.

3. Реалізація

Розв'язок оформлено одним файлом `solver.py` (Додаток А), сумісним із Python 2.7 (саме він у Docker-образі) і Python 3. Основні елементи:

- `gen_block(seed, idx, n)` генерує друкований текст довжини n для блоку idx детерміновано (Mersenne Twister), тому майстер і воркери отримують однакові дані незалежно від того, хто і скільки блоків генерує.
- Шифр Цезаря реалізовано через таблицю перекодування байтів і `bytes.translate()`, тобто цикл по байтах виконується в C. XOR виконується через довгу арифметику Python: текст і потік ключа переводяться у великі цілі числа й XOR-яться одним виразом. Це дозволяє працювати на Python 2.7 без NumPy, якого в образі немає, і значно швидше за цикл у Python.
- `process_blocks()` обробляє діапазон блоків на одному вузлі й повертає таймінги за етапами (генерація, Цезар, XOR, перевірка), результат перевірки та md5 кожного зашифрованого блоку. Цей самий код виконується в режимі `serial` на майстрі й у режимі `local` на воркерах через віддалений метод `Solver.process_range()`.
- Режим `scatter` використовує віддалені методи `caesar_chunk()` і `xor_chunk()`, які приймають шматок тексту у base64, обробляють його й повертають також у base64. Розподіл шматків повторює `parallel.cpp`: `split_counts()` обчислює $count_i = rest/(p-i)$ та зміщення.
- Для кожного обсягу програма виводить таблицю з часом, відбитками md5 і рядки `CSV,...`, які далі обробляє `report.py`.

Коректність перевірено двома способами: розшифрований текст порівнюється з вихідним на кожному запуску, а md5-відбитки шифротексту збігаються в усіх режимах і для будь-якої кількості воркерів (наприклад, для 100 МБ відбиток Цезаря `4e0e8be3e8bb67b6b0ddf7db719f618e` отримано і в `serial`, і в `local` на 1--4 воркерах, і в `scatter`). Локально, без хмари, програму можна запустити командою `python solver.py inputs/test_small.txt out.txt --workers 2`: воркери імітуються окремими процесами. Перед розгортанням розв'язок було також прогнано на справжньому PARCS-Python, піднятому локально (Python 2.7.18, Pyro4 4.75, майстер і 3 воркери).

Допоміжні скрипти: `cluster.sh` створює й видаляє кластер у Compute Engine, `run_all.sh` надсилає всі вхідні файли на майстер через його HTTP API і збирає output-файли, `report.py` будує таблиці прискорення й ефективності та графіки.

4. Розгортання системи ПАРКС у Google Cloud Platform

Систему розгорнуто у проєкті Google Cloud `project-abae6256-cc9d-4a12-b5b` (Free Trial), регіон Європа (Фінляндія), зона `europe-north1-b`. Створено п'ять віртуальних машин: один master і чотири воркери. Параметри середовища наведено в таблиці 1, вкладка Nodes із чотирма зареєстрованими воркерами показана на рисунку 1.

Таблиця 1 – Параметри тестового середовища

Параметр	Значення
Платформа	Google Compute Engine, зона europe-north1-b
Кількість вузлів	5 (1 master + 4 воркери)
Тип ВМ	n1-standard-1: 1 vCPU, 3,75 ГБ RAM (веб-інтерфейс ПАРКС показує 4 GB)
Процесор	Intel Xeon @ 2.00 GHz
ОС вузлів	Container-Optimized OS (cos-stable), Docker вбудований
Образ ПАРКС	hummer12007/parcs-node (Docker Hub); Python 2.7 у контейнері, Pyro4
Мережа	внутрішня мережа VPC 10.166.0.0/20; веб-інтерфейс master на порту 8080 відкрито лише для IP Cloud Shell і власного IP
Керування кількістю воркерів	параметр `workers=` у вхідному файлі (кластер піднімається один раз)

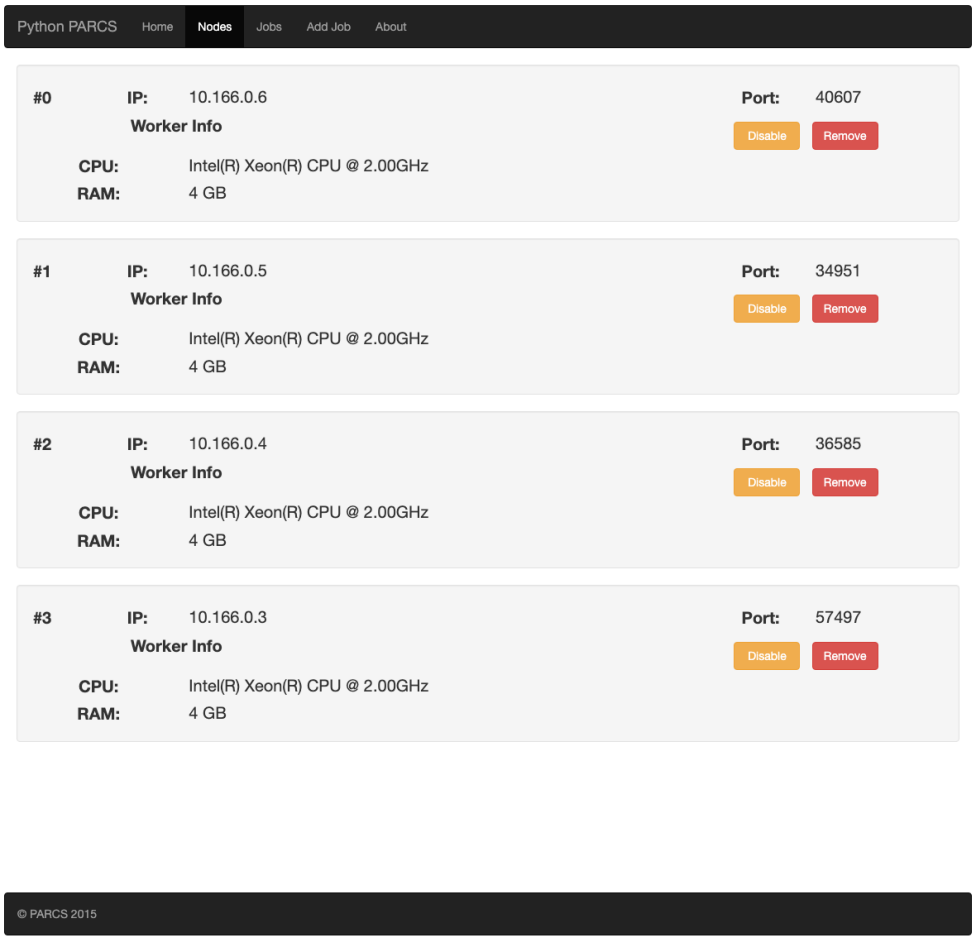


Рисунок 1 – Вкладка Nodes: чотири зареєстровані воркери (10.166.0.3--10.166.0.6)

Розгортання виконується скриптом `cluster.sh` у Cloud Shell (повна інструкція в Додатку Б):

1. вмикається Compute Engine API; створюється правило фаєрвола `parcs-web`, яке відкриває порт 8080 лише для IP-адреси Cloud Shell і адреси користувача (правило `allow-all` з офіційного гайду свідомо не використовується, бо через веб-інтерфейс ПАРКС можна виконати довільний код на VM);
2. створюється VM `master` зі startup-скриптом, який запускає контейнер `docker run --network=host -e PARCS_ARGS="master" hummer12007/parcs-node`;
3. створюються VM `worker1`--`worker4` з аналогічним скриптом і `PARCS_ARGS="worker <внутрішня IP master>"`;
4. скрипт чекає, поки всі воркери зареєструються на майстрі (`/api/worker`), і друкує адресу веб-інтерфейсу.

Під час розгортання виникли особливості, які відрізняються від базової інструкції PARCS-Python:

- команда `create-with-container` (підкоманда `gcloud compute instances`) з офіційного гайду позначена в gcloud як deprecated, тому використано звичайні VM зі startup-скриптом, що запускає контейнер з параметром `--network=host`;

- у зоні `europe-north1-a` не було вільних машин типу `n1-standard-1` (помилка `ZONE_RESOURCE_POOL_EXHAUSTED`), тому всі вузли створено в сусідній зоні `europe-north1-b`;
- Container-Optimized OS має власний host-firewall, який пропускає лише SSH: воркери не могли під'єднатися до майстра (`Connection timed out` на `10.166.0.2:8080`). У startup-скрипт додано правила `iptables`, що дозволяють TCP із внутрішньої мережі (Pyro4 і Flask на воркерах слухають випадкові порти) та порт 8080; для вже створених VM є команда `cluster.sh fix-firewall`;
- в образі працює Python 2.7, тому у файлі розв'язку оголошено кодування `utf-8` і уникнуто конструкцій Python 3 (f-рядки, анотації типів).

5. Експерименти

Усі 10 задач надіслано на майстер скриптом `run_all.sh` і виконано по черзі (рисунок 2). Для режиму `local` використано обсяги 100 МБ, 500 МБ і 1 ГБ, для `scatter` 10, 50 і 100 МБ, бо майстер тримає в пам'яті весь текст і кілька його копій. Кожен замір виконано один раз; усі запуски дали `PASSED`, відбитки збіглися.

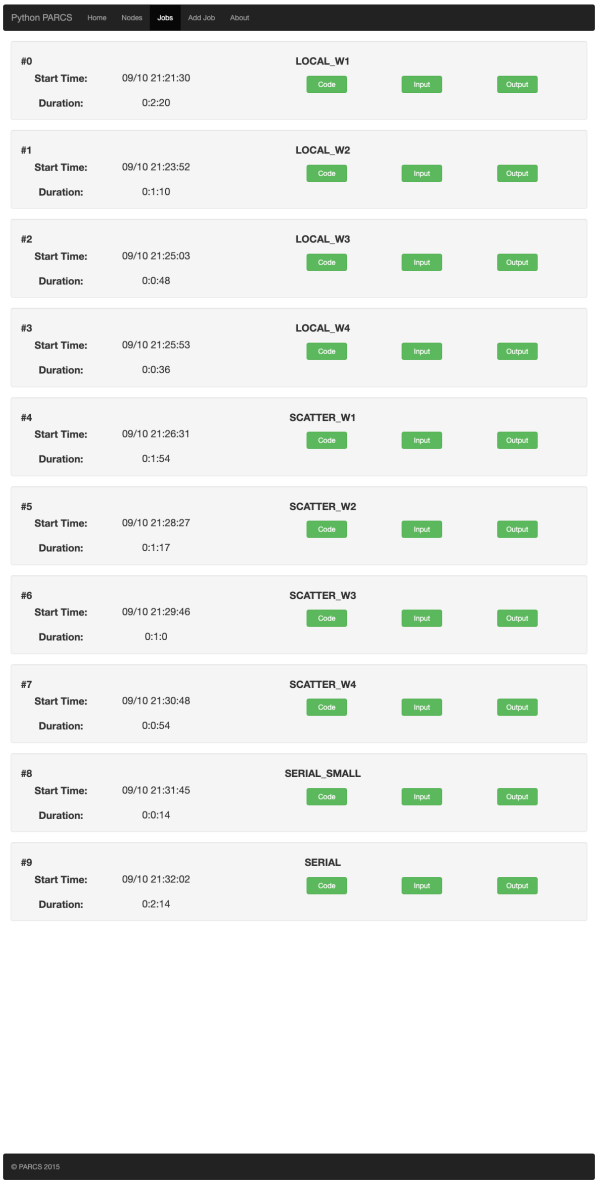


Рисунок 2 – Вкладка Jobs: 10 задач, виконаних на кластері

5.1. Режим local

Таблиця 2 – Час виконання, прискорення та ефективність у режимі local

Розмір	Воркери	Час, с	Прискорення S	Ефективність E, %
100 МБ	послідовний	8,180	1,00	100
100 МБ	1	8,832	0,93	93
100 МБ	2	4,405	1,86	93
100 МБ	3	3,035	2,70	90
100 МБ	4	2,387	3,43	86
500 МБ	послідовний	41,783	1,00	100
500 МБ	1	43,934	0,95	95
500 МБ	2	22,164	1,89	94

Розмір	Воркери	Час, с	Прискорення S	Ефективність E, %
500 МБ	3	14,768	2,83	94
500 МБ	4	11,335	3,69	92
1 ГБ	послідовний	83,989	1,00	100
1 ГБ	1	86,882	0,97	97
1 ГБ	2	43,136	1,95	97
1 ГБ	3	29,832	2,82	94
1 ГБ	4	22,234	3,78	94

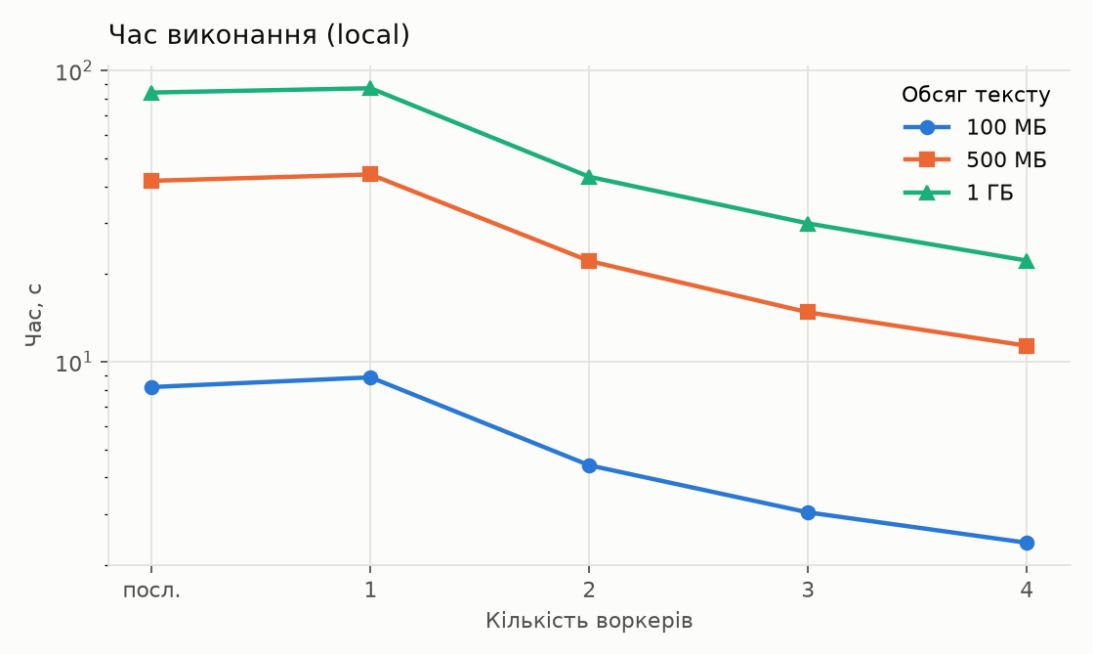


Рисунок 3 – Час виконання залежно від кількості воркерів (local)

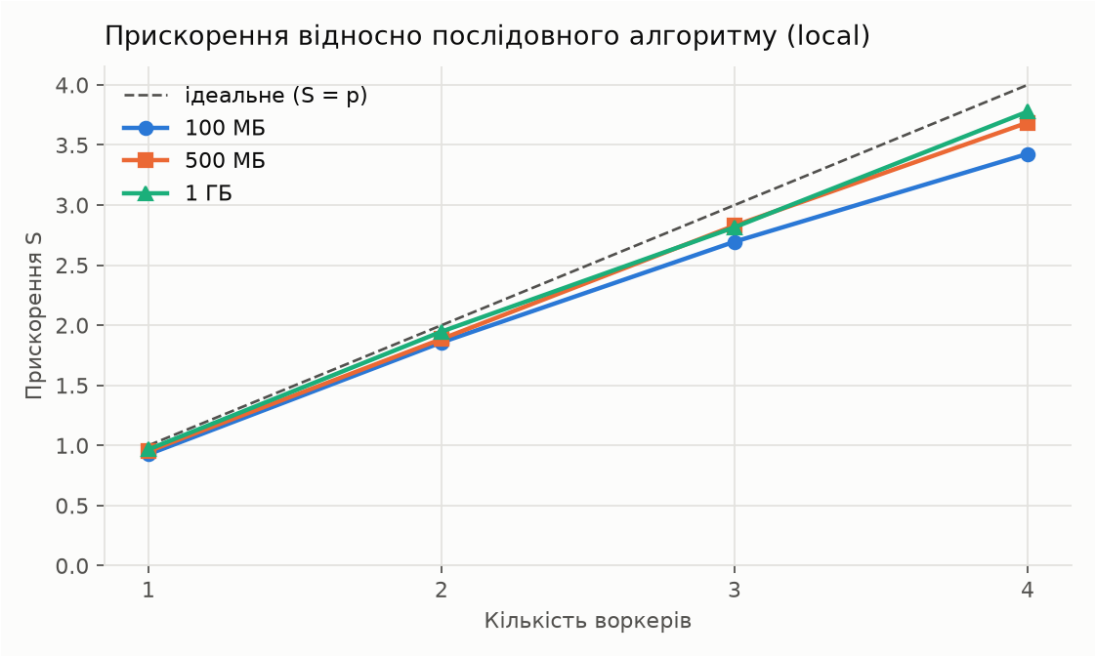


Рисунок 4 – Прискорення відносно послідовного алгоритму (local)

Розподіл часу на одному вузлі для 1 ГБ у послідовному режимі: генерація тексту 25,5 с, XOR 51,9 с, шифр Цезаря 1,6 с, решта перевірка. Отже, на один байт припадає близько 25 нс генерації, 52 нс XOR і лише 1,5 нс Цезаря (`bytes.translate` виконується в C).

5.2. Режим scatter

Таблиця 3 – Час виконання, прискорення та ефективність у режимі scatter (база: Цезар + XOR у послідовному режимі)

Розмір	Воркери	Час, с	Прискорення S	Ефективність E, %
10 МБ	послідовний	0,536	1,00	100
10 МБ	1	6,629	0,08	8
10 МБ	2	3,694	0,14	7
10 МБ	3	2,744	0,20	7
10 МБ	4	2,268	0,24	6
50 МБ	послідовний	2,796	1,00	100
50 МБ	1	34,503	0,08	8
50 МБ	2	22,441	0,12	6
50 МБ	3	16,945	0,17	6
50 МБ	4	14,916	0,19	5
100 МБ	послідовний	5,204	1,00	100
100 МБ	1	67,481	0,08	8
100 МБ	2	45,277	0,11	6
100 МБ	3	34,930	0,15	5
100 МБ	4	31,802	0,16	4

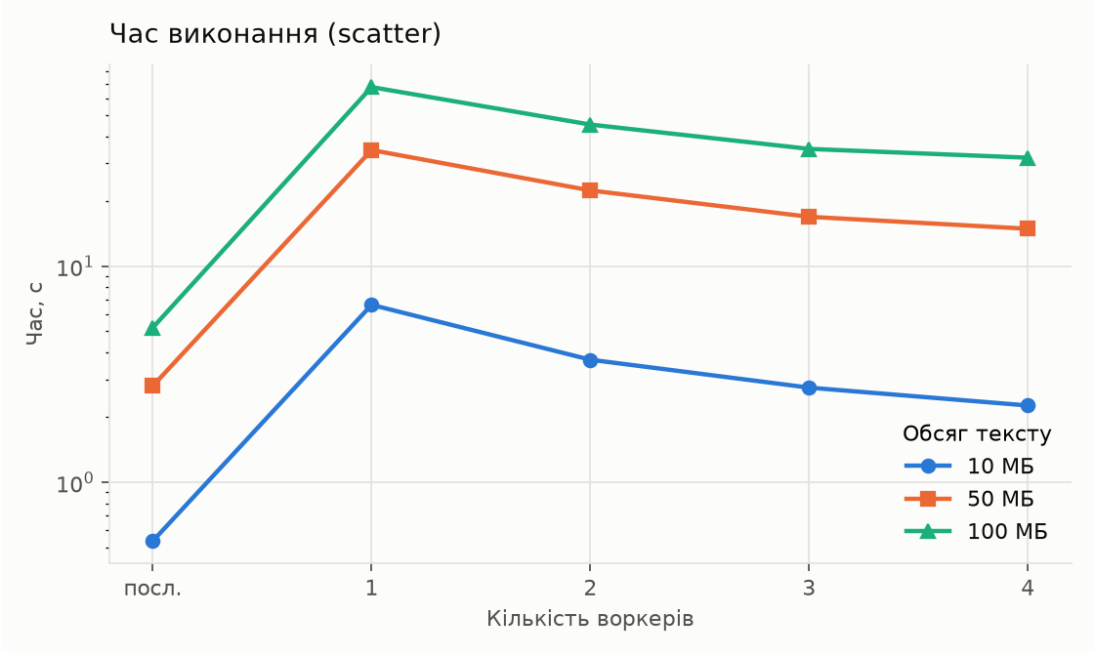


Рисунок 5 – Час виконання залежно від кількості воркерів (scatter), логарифмічна шкала

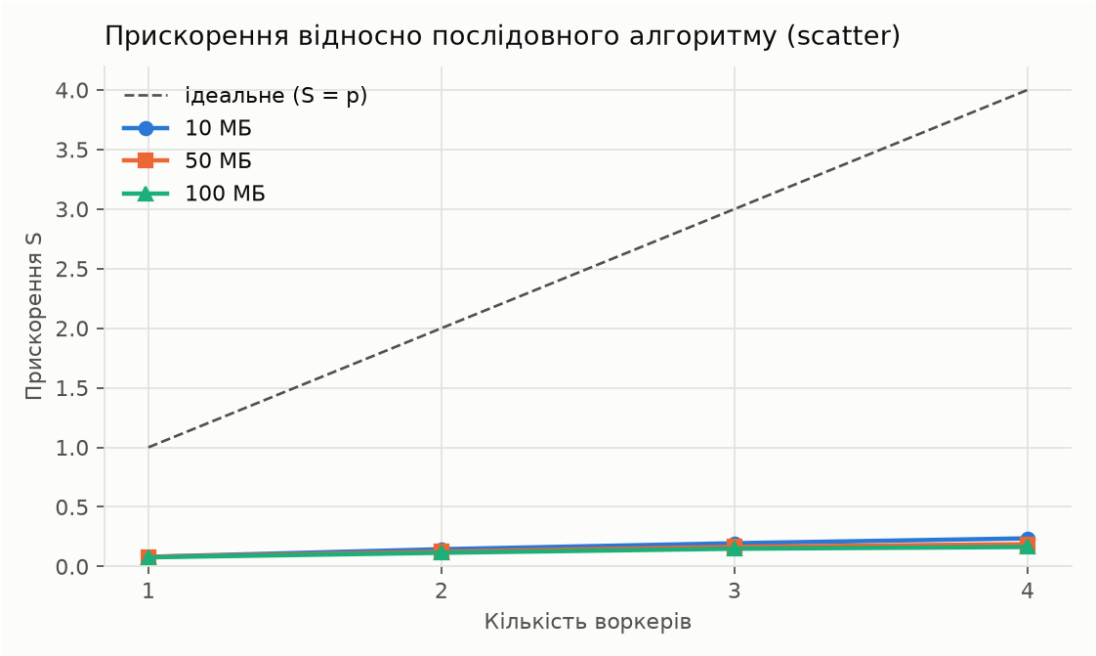


Рисунок 6 – Прискорення відносно послідовного алгоритму (scatter)

6. Аналіз результатів

Режим local. Прискорення близьке до лінійного: для 1 ГБ на чотирьох воркерах $S = 3{,}78$ при ефективності 94 %. Накладні витрати на виклики Pyro4 становлять близько 10 мс на запуск (стовпець Overhead у вихідних файлах), тобто менше 0,1 % часу. Втрати відносно ідеального прискорення мають дві причини. По-перше, час усього запуску визначає найповільніший воркер: окремі VM відрізняються на кілька відсотків (запуск на одному воркері повільніший за послідовний на майстрі на 3--7 %). По-друге, для 100 МБ частка фіксованих витрат більша,

тому ефективність нижча (86 %), а зі збільшенням обсягу вона зростає до 94 %: це класична поведінка за законом Густафсона, коли паралельна частина росте разом із задачею.

Режим scatter. Паралельна версія повільніша за послідовну в усіх конфігураціях: $S = 0{,}08$ на одному воркері та лише $0{,}16$ на чотирьох. Причина в обчислювальній інтенсивності задачі. На кожен байт припадає 1,5 нс обчислень для Цезаря і 52 нс для XOR, тоді як у режимі `scatter` кожен байт чотири рази проходить через мережу туди й назад (шифрування і розшифрування для кожного з двох шифрів), до того ж у base64 (+33 % обсягу) і через серіалізацію Pyro4. Для 100 МБ через майстер проходить близько 800 МБ даних, і на одному воркері це займає 62 с понад час обчислень, тобто приблизно 78 нс на байт на кожну передачу. Передача дорожча за обчислення на один-два порядки, тому задача є комунікаційно обмеженою. Збільшення кількості воркерів допомагає лише частково (67 с → 32 с), бо паралелізуються тільки приймання, кодування і обчислення на воркерах, а майстер залишається вузьким місцем: він один серіалізує, передає й приймає всі дані через одну мережеву карту й один vCPU. Це закон Амдала з великою послідовною частиною.

Порівняння режимів. Обидва режими дають однакові md5-відбитки шифротексту, тобто реалізують один і той самий алгоритм; відрізняється лише те, де лежать дані. Коли дані вже є на вузлах (`local`), шифрування масштабується майже ідеально. Коли дані треба роздати з одного вузла (`scatter`), для таких дешевих операцій, як Цезар і XOR, вигідніше взагалі не паралелити. Це й підтверджує висновок MPI-версії: розподілені обчислення виграють тоді, коли обчислень на байт даних багато або коли дані розподілені заздалегідь.

Висновки

1. Реалізовано паралельне шифрування та розшифрування шифром Цезаря і XOR для платформи PARCS-Python у трьох режимах: послідовному, `local` (дані генеруються на воркерах) і `scatter` (порт MPI-версії з розсиланням даних з майстра). Коректність підтверджено перевіркою розшифрованого тексту й збігом md5-відбитків шифротексту в усіх режимах.
2. Систему ПАРКС розгорнуто в Google Cloud Platform: master і 4 воркери на VM `n1-standard-1` у зоні `europe-north1-b`, Docker-образ `hummer12007/parcs-node`, автоматизовано скриптом `cluster.sh`. Виявлено і подолано три особливості: застарілу команду `create-with-container`, брак ресурсів у зоні `europe-north1-a` та host-firewall Container-Optimized OS, що блокував з'єднання воркерів з майстром.
3. У режимі `local` отримано прискорення 1,95, 2,82 і 3,78 для 2, 3 і 4 воркерів на 1 ГБ тексту при ефективності 94--97 %, що близько до лінійного.
4. У режимі `scatter` паралельна версія повільніша за послідовну (прискорення 0,08--0,24), бо передача байта між вузлами коштує на порядок більше за його шифрування. Задача є комунікаційно обмеженою, а майстер залишається вузьким місцем незалежно від кількості воркерів.
5. Загальний висновок: ефективність паралельної програми визначає не кількість вузлів,

а відношення обчислень до обміну даними. Для задач з низькою обчислювальною інтенсивністю дані мають бути розподілені заздалегідь, інакше паралелізм не дає виграшу.

Література

1. Дерев'янченко О. В. Система ПАРКС-JAVA для паралельних обчислень на комп'ютерних мережах: навчальний посібник. Київ: КНУ імені Тараса Шевченка.
2. PARCS-Python: `https://github.com/Hummer12007/parcs-python`.
3. Документація Google Compute Engine: `https://cloud.google.com/compute/docs`.
4. Container-Optimized OS, host firewall: `https://cloud.google.com/container-optimized-os/docs/how-to/create-configure-instance`.
5. Репозиторій роботи: `https://github.com/altiebalti/parcs-caesar-xor`.

Додаток А. Файл розв'язку solver.py

```
# -*- coding: utf-8 -*-
"""
PARCS-Python: паралельне шифрування шифром Цезаря та XOR.Порт

 MPIверсії- (serial.cpp / parallel.cpp) на платформу PARCS-Python
(https://github.com/Hummer12007/parcs-python). Сумісний з Python 2.7саме
( він у Dockerобразі- hummer12007/parcs-node) і з Python 3.Режими

 рядок( mode=... у вхідному файлі):
  serial  - послідовний алгоритм, усе рахує майстер, воркери не використовуються;
  local   - кожен воркер сам генерує свою частину тексту дані( "лежать" на вузлі),
            шифруєрозшифровує/ її і повертає лише контрольні суми;
  scatter - дослівний порт MPIверсії-: майстер генерує весь текст, розсилає шматки
            воркерам як( MPI_Scatterv), збирає результат як( MPI_Gatherv).Текст

 генерується блоками фіксованого розміру, кожен блок - зі свого зерна
(seed, номер блоку), тому дані однакові за будьякої- кількості воркерів, а
"відбитки" шифротексту (fp) мають збігатися в усіх режимах.Локальний

 запуск воркери( імітуються процесами):
    python solver.py input.txt output.txt --workers 4
"""
from __future__ import print_function, division

import base64
import binascii
import hashlib
import random
import sys
import time

try:
    from Pyro4 import expose
except ImportError:  # локальний запуск без Pyro4
    def expose(f):
        return f

PY2 = sys.version_info[0] == 2
if PY2:
    range = xrange  # noqa: F821

DEFAULTS = {
    'mode': 'local',
    'sizes': '100000000',
    'workers': '0',          # 0 = усі воркери, що є в PARCS
    'shift': '3',
    'key': 'SecretKey123!@#',
```

```
    'seed': '2026',
    'block': '1048576',       # розмір блоку генераціїобробки/, байт
    'repeat': '1',            # скільки разів повторити кожен замір
}


# ---------------------------------------------------------------------------
# Базові операції виконуються( і на майстрі, і на воркерах)
# ---------------------------------------------------------------------------

def make_table(fn):
    """Таблиця перекодування байтів 0..255 для str/bytes.translate()."""
    return bytes(bytearray(fn(b) % 256 for b in range(256)))


PRINTABLE = make_table(lambda b: 32 + b % 95)    # байт -> друкований символ 32..126


def caesar_tables(shift):
    """Таблиці шифрування та розшифрування Цезаря: (b + shift) % 256 і (b - shift) % 256."""
    return make_table(lambda b: b + shift), make_table(lambda b: b - shift)


def gen_block(seed, idx, n):
    """Випадковий друкований текст довжини n для блоку idx детерміновано(, Mersenne Twister).
    """
    if n <= 0:
        return b''
    rng = random.Random(seed * 1000003 + idx)
    raw = binascii.unhexlify(format(rng.getrandbits(8 * n), '0%dx' % (2 * n)))
    return raw.translate(PRINTABLE)


def keystream(key, offset, n):
    """Ключ XOR, повторений до довжини n, починаючи з глобальної позиції offset."""
    s = offset % len(key)
    rotated = key[s:] + key[:s]
    return (rotated * (n // len(key) + 1))[:n]


def xor_bytes(data, key, offset):
    """data[i] ^ key[(offset + i) % len(key)] для всього шматка через( довгу арифметику)."""
    n = len(data)
    if n == 0:
        return data
    x = int(binascii.hexlify(data), 16) ^ int(binascii.hexlify(keystream(key, offset, n)),
    16)
    return binascii.unhexlify(format(x, '0%dx' % (2 * n)))
```

```
def md5hex(data):
    return hashlib.md5(data).hexdigest()


def fingerprint(digests):
    """Один відбиток для всього шифротексту з md5 окремих блоків у( порядку блоків)."""
    return md5hex(''.join(digests).encode('ascii'))


def to_wire(data):
    """bytes -> ASCIIрядок- для передачі через Pyro4 серіалізатор( serpent)."""
    s = base64.b64encode(data)
    return s if PY2 else s.decode('ascii')


def from_wire(s):
    if not isinstance(s, bytes):
        s = s.encode('ascii')
    return base64.b64decode(s)


def as_bytes(s):
    if isinstance(s, bytes):
        return s
    return s.encode('latin-1')


def value(result):
    """Результат асинхронного виклику Pyro4 (FutureResult.value) або звичайне значення."""
    return result.value if hasattr(result, 'value') else result


def split_counts(total, parts):
    """Як у parallel.cpp: count_i = rest / (parts - i); повертає (counts, displs)."""
    counts, displs, rest, pos = [], [], total, 0
    for i in range(parts):
        c = rest // (parts - i)
        counts.append(c)
        displs.append(pos)
        rest -= c
        pos += c
    return counts, displs


def process_blocks(seed, first, count, block, size, shift, key):
    """Згенерувати блоки [first, first+count), зашифрувати і розшифрувати Цезарем та XOR,
    перевірити результат. Повертає статистику і md5 кожного зашифрованого блоку."""
    enc_t, dec_t = caesar_tables(shift)
```

```
    t = {'gen': 0.0, 'caesar': 0.0, 'xor': 0.0, 'verify': 0.0}
    caesar_ok = xor_ok = True
    c_digests, x_digests = [], []
    nbytes = 0
    start = time.time()
    for idx in range(first, first + count):
        offset = idx * block
        n = min(block, size - offset)
        nbytes += n

        t0 = time.time()
        text = gen_block(seed, idx, n)
        t1 = time.time()
        c_enc = text.translate(enc_t)
        c_dec = c_enc.translate(dec_t)
        t2 = time.time()
        x_enc = xor_bytes(text, key, offset)
        x_dec = xor_bytes(x_enc, key, offset)
        t3 = time.time()
        caesar_ok = caesar_ok and c_dec == text
        xor_ok = xor_ok and x_dec == text
        c_digests.append(md5hex(c_enc))
        x_digests.append(md5hex(x_enc))
        t4 = time.time()

        t['gen'] += t1 - t0
        t['caesar'] += t2 - t1
        t['xor'] += t3 - t2
        t['verify'] += t4 - t3
    t['total'] = time.time() - start
    return {'first': first, 'count': count, 'bytes': nbytes,
            'caesar_ok': caesar_ok, 'xor_ok': xor_ok,
            'caesar_digests': c_digests, 'xor_digests': x_digests, 'times': t}


# ------------------------------------------------------------------------
# Solver - клас, який завантажує PARCS на( майстрі з воркерами, на воркерах без аргументів)
# ------------------------------------------------------------------------

class Solver:
    def __init__(self, workers=None, input_file_name=None, output_file_name=None):
        self.input_file_name = input_file_name
        self.output_file_name = output_file_name
        self.workers = workers

    # ---------------- майстер ----------------

    def solve(self):
        cfg = self.read_input()
```

```
        workers = list(self.workers or [])
        if cfg['workers'] > 0:
            workers = workers[:cfg['workers']]
        mode = cfg['mode']
        if mode != 'serial' and not workers:
            raise ValueError('mode=%s needs at least one worker' % mode)
        print('Job started: mode=%s workers=%d sizes=%s' % (mode, len(workers), cfg['sizes'])
)

        rows = []
        for size in cfg['sizes']:
            runs = []
            for _ in range(cfg['repeat']):
                if mode == 'serial':
                    runs.append(self.run_serial(cfg, size))
                elif mode == 'local':
                    runs.append(self.run_local(cfg, size, workers))
                elif mode == 'scatter':
                    runs.append(self.run_scatter(cfg, size, workers))
                else:
                    raise ValueError('unknown mode: %s' % mode)
            best = min(runs, key=lambda r: r['wall'])
            best['wall_avg'] = sum(r['wall'] for r in runs) / len(runs)
            best['repeat'] = len(runs)
            rows.append(best)
            print('size=%d wall=%.3f s ok=%s' % (size, best['wall'], best['caesar_ok'] and
best['xor_ok']))

        self.write_output(format_report(cfg, rows))
        print('Job finished')

    def run_serial(self, cfg, size):
        """Послідовний алгоритм: той самий конвеєр, що в режимі local, але на одній машині.""
"
        nblocks = -(-size // cfg['block'])
        start = time.time()
        r = process_blocks(cfg['seed'], 0, nblocks, cfg['block'], size, cfg['shift'], cfg['
key'])
        wall = time.time() - start
        return self.summary('serial', 1, size, wall, [r])

    def run_local(self, cfg, size, workers):
        """Кожен воркер обробляє свій діапазон блоків; по мережі йдуть лише параметри і
контрольні суми."""
        k = len(workers)
        nblocks = -(-size // cfg['block'])
        bounds = [nblocks * i // k for i in range(k + 1)]
        key_wire = to_wire(cfg['key'])
        start = time.time()
```

```
        futures = []
        for i in range(k):
            count = bounds[i + 1] - bounds[i]
            if count > 0:
                futures.append(workers[i].process_range(cfg['seed'], bounds[i], count, cfg['
block'],
                                                        size, cfg['shift'], key_wire))
        parts = [value(f) for f in futures]
        wall = time.time() - start
        return self.summary('local', k, size, wall, parts)

    def run_scatter(self, cfg, size, workers):
        """Порт parallel.cpp: Scatterv -> шифрування -> Gatherv -> Scatterv -> розшифрування
-> Gatherv."""
        k = len(workers)
        block, seed, shift, key = cfg['block'], cfg['seed'], cfg['shift'], cfg['key']
        nblocks = -(-size // block)

        # генерація даних на майстрі як( і в MPIверсії-, не входить у виміряний час)
        t0 = time.time()
        text = b''.join(gen_block(seed, i, min(block, size - i * block)) for i in range(
nblocks))
        t_gen = time.time() - t0

        counts, displs = split_counts(size, k)
        key_wire = to_wire(key)

        def scatter_gather(method, data, extra):
            futures = [getattr(workers[i], method)(to_wire(data[displs[i]:displs[i] + counts[
i]]),
                                                   *extra(i))
                       for i in range(k)]
            return b''.join(from_wire(value(f)) for f in futures)

        start = time.time()
        c_enc = scatter_gather('caesar_chunk', text, lambda i: (shift, False))
        c_dec = scatter_gather('caesar_chunk', c_enc, lambda i: (shift, True))
        t_caesar = time.time() - start

        start = time.time()
        x_enc = scatter_gather('xor_chunk', text, lambda i: (key_wire, displs[i]))
        x_dec = scatter_gather('xor_chunk', x_enc, lambda i: (key_wire, displs[i]))
        t_xor = time.time() - start

        # перевірка і відбитки не( входять у виміряний час, як і в MPIверсії-)
        t0 = time.time()
        part = {
            'first': 0, 'count': nblocks, 'bytes': size,
            'caesar_ok': c_dec == text, 'xor_ok': x_dec == text,
```

```
        'caesar_digests': [md5hex(c_enc[i * block:(i + 1) * block]) for i in range(
nblocks)],
        'xor_digests': [md5hex(x_enc[i * block:(i + 1) * block]) for i in range(nblocks)
],
        'times': {'gen': t_gen, 'caesar': t_caesar, 'xor': t_xor, 'verify': 0.0,
                  'total': t_caesar + t_xor},
    }
    part['times']['verify'] = time.time() - t0
    return self.summary('scatter', k, size, t_caesar + t_xor, [part])

@staticmethod
def summary(mode, workers, size, wall, parts):
    parts = sorted(parts, key=lambda p: p['first'])
    times = [p['times'] for p in parts]
    slowest = max(times, key=lambda t: t['total'])
    c_digests, x_digests = [], []
    for p in parts:
        c_digests.extend(p['caesar_digests'])
        x_digests.extend(p['xor_digests'])
    return {
        'mode': mode, 'workers': workers, 'size': size, 'wall': wall,
        'caesar': slowest['caesar'], 'xor': slowest['xor'],
        'gen': slowest['gen'], 'verify': slowest['verify'],
        'compute': slowest['total'],
        'overhead': max(0.0, wall - slowest['total']) if mode == 'local' else 0.0,
        'bytes': sum(p['bytes'] for p in parts),
        'caesar_ok': all(p['caesar_ok'] for p in parts),
        'xor_ok': all(p['xor_ok'] for p in parts),
        'fp_caesar': fingerprint(c_digests), 'fp_xor': fingerprint(x_digests),
    }

# ---------------- воркер ----------------

@staticmethod
@expose
def process_range(seed, first, count, block, size, shift, key_wire):
    return process_blocks(seed, first, count, block, size, shift, from_wire(key_wire))

@staticmethod
@expose
def caesar_chunk(chunk_wire, shift, decrypt):
    enc_t, dec_t = caesar_tables(shift)
    return to_wire(from_wire(chunk_wire).translate(dec_t if decrypt else enc_t))

@staticmethod
@expose
def xor_chunk(chunk_wire, key_wire, offset):
    return to_wire(xor_bytes(from_wire(chunk_wire), from_wire(key_wire), offset))
```

```
    # ---------------- ввід / вивід ----------------

    def read_input(self):
        cfg = dict(DEFAULTS)
        with open(self.input_file_name) as f:
            for raw in f:
                line = raw.strip()
                if not line or line.startswith('#') or '=' not in line:
                    continue
                name, val = line.split('=', 1)
                name = name.strip().lower()
                # у ключі XOR може бути '#', тому коментарі після значення знімаємо для всіх,
     крім key
                cfg[name] = val.strip() if name == 'key' else val.split('#', 1)[0].strip()
        out = {
            'mode': cfg['mode'].lower(),
            'sizes': [int(s) for s in cfg['sizes'].replace(',', ' ').split()],
            'workers': int(cfg['workers']),
            'shift': int(cfg['shift']),
            'key': as_bytes(cfg['key']),
            'seed': int(cfg['seed']),
            'block': int(cfg['block']),
            'repeat': max(1, int(cfg['repeat'])),
        }
        if not out['key']:
            raise ValueError('key must not be empty')
        return out

    def write_output(self, text):
        with open(self.output_file_name, 'w') as f:
            f.write(text)


def format_report(cfg, rows):
    lines = []
    head = rows[0]
    lines.append('=== Caesar / XOR on PARCS-Python ===')
    lines.append('mode=%s  workers=%d  shift=%d  key_len=%d  block=%d  repeat=%d'
                 % (head['mode'], head['workers'], cfg['shift'], len(cfg['key']), cfg['block'
    ], cfg['repeat']))
    lines.append('')
    fmt = '%-12s | %-10s | %-10s | %-10s | %-10s | %-10s | %-10s | %-6s'
    lines.append(fmt % ('Data size', 'Time, s', 'Avg, s', 'Caesar, s', 'XOR, s', 'Gen, s', '
    Overhead,s', 'Check'))
    lines.append('-' * 100)
    for r in rows:
        lines.append(fmt % (r['size'], '%.3f' % r['wall'], '%.3f' % r['wall_avg'], '%.3f' % r
    ['caesar'],
                            '%.3f' % r['xor'], '%.3f' % r['gen'], '%.3f' % r['overhead'],
```

```
                          'PASSED' if r['caesar_ok'] and r['xor_ok'] else 'FAILED'))
    lines.append('')
    lines.append('Fingerprints (must be identical in every mode for the same size):')
    for r in rows:
        lines.append('  %-12d caesar=%s  xor=%s' % (r['size'], r['fp_caesar'], r['fp_xor']))
    lines.append('')
    lines.append('# CSV: mode,workers,size,time,time_avg,caesar,xor,gen,overhead,check')
    for r in rows:
        lines.append('CSV,%s,%d,%d,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f,%s'
                     % (r['mode'], r['workers'], r['size'], r['wall'], r['wall_avg'], r['
    caesar'], r['xor'],
                        r['gen'], r['overhead'], 'PASSED' if r['caesar_ok'] and r['xor_ok']
    else 'FAILED'))
    lines.append('')
    return '\n'.join(lines)


# ---------------------------------------------------------------------------
# Локальний запуск без PARCS: воркери імітуються окремими процесами
# ---------------------------------------------------------------------------

def _call(name, args):
    return getattr(Solver, name)(*args)


class _LocalFuture(object):
    def __init__(self, async_result):
        self._r = async_result

    @property
    def value(self):
        return self._r.get()


class _LocalWorker(object):
    def __init__(self, pool):
        self._pool = pool

    def __getattr__(self, name):
        def call(*args):
            return _LocalFuture(self._pool.apply_async(_call, (name, args)))
        return call


def main(argv):
    import argparse
    import multiprocessing
    parser = argparse.ArgumentParser(description='Local run of the PARCS solver (workers =
    processes)')
```

```
    parser.add_argument('input')
    parser.add_argument('output')
    parser.add_argument('--workers', type=int, default=2)
    args = parser.parse_args(argv)
    pools = [multiprocessing.Pool(1) for _ in range(args.workers)]
    try:
        Solver([_LocalWorker(p) for p in pools], args.input, args.output).solve()
    finally:
        for p in pools:
            p.terminate()
    with open(args.output) as f:
        print(f.read())


if __name__ == '__main__':
    main(sys.argv[1:])
```

Додаток Б. Інструкція з розгортання та запуску

Репозиторій проєкту: `https://github.com/altiebalti/parcs-caesar-xor`. Структура: `solver.py` --- розв'язок для ПАРКС; `inputs/` --- вхідні файли для всіх запусків; `cluster.sh` --- створення й видалення кластера в Compute Engine; `run_all.sh` --- надсилання задач на майстер і збирання результатів; `report.py` --- таблиці та графіки; `outputs/` --- вихідні файли запусків, `results.csv`, `summary.md` і графіки; `report/` --- цей звіт і рисунки.

Б.1. Підготовка Google Cloud

Потрібен проєкт з активним Free Trial. Усі команди виконуються в Cloud Shell (кнопка Activate Cloud Shell у консолі), де `gcloud` уже встановлений і залогінений.

```
git clone https://github.com/altiebalti/parcs-caesar-xor.git
cd parcs-caesar-xor
gcloud config set project <PROJECT_ID>
```

Б.2. Створення кластера

```
export ZONE=europe-north1-b        # за замовчуванням europe-north1-a
MY_IPваш=< IP з https://ifconfig.me> bash cluster.sh up 4
```

Скрипт вмикає Compute Engine API, створює правило фаєрвола `parcs-web` (порт 8080 лише для Cloud Shell і `MY_IP`), VM `master` і `worker1`--`worker4` типу `n1-standard-1` на Container-Optimized OS зі startup-скриптом, який відкриває порти в host-firewall і запускає контейнер `hummer12007/parcs-node` у мережі хоста. Далі він чекає реєстрації воркерів і друкує адресу `http://<IP майстра>:8080`. Якщо зона не має вільних машин (`ZONE_RESOURCE_POOL_EXHAUSTED`), змініть `ZONE` і повторіть команду. Інші команди: `status`, `ip`, `allow` (оновити IP у фаєрволі), `fix-firewall` (відкрити порти на вже створених VM), `stop`/`start`, `down`.

Startup-скрипт вузла:

```
#!/bin/bash
iptables -w -I INPUT -p tcp -s 10.0.0.0/8 -j ACCEPT
iptables -w -I INPUT -p tcp --dport 8080 -j ACCEPT
docker start parcs 2>/dev/null || \
  docker run -d --name parcs --restart=always --network=host \
    -e PARCS_ARGS="worker внутрішня< IP master>" hummer12007/parcs-node
```

Б.3. Запуск експериментів

```
bash run_all.sh $(bash cluster.sh ip)       # усі 10 задач по черзі, результати в outputs/
pip3 install --user matplotlib
python3 report.py outputs/                  # summary.md, results.csv, графіки
```

`run_all.sh` надсилає кожен вхідний файл разом із `solver.py` на `POST /api/job` майстра і чекає, поки `GET /api/job/<id>/output` поверне результат. Те саме можна зробити вручну через веб-інтерфейс: Add Job, файл розв'язку `solver.py`, вхідний файл з `inputs/`.

Формат вхідного файлу:

```
mode=local                              # serial | local | scatter
workers=4                               # скільки воркерів узяти (0 = усі)
sizes=100000000, 500000000, 1000000000  # обсяги тексту в байтах
shift=3                                 # зсув Цезаря
key=SecretKey123!@#                     # ключ XOR
seed=2026                               # зерно генератора
```

Б.4. Видалення ресурсів

```
bash cluster.sh down     # видаляє VM і правило фаєрвола
```

Додаток В. Приклад вихідного файлу (local_w4.txt)

```
=== Caesar / XOR on PARCS-Python ===
mode=local  workers=4  shift=3  key_len=15  block=1048576  repeat=1

Data size    | Time, s    | Avg, s     | Caesar, s  | XOR, s     | Gen, s     | Overhead,s |
    Check
-------------------------------------------------------------------------------------------------------

100000000    | 2.387      | 2.387      | 0.044      | 1.475      | 0.737      | 0.009      |
    PASSED
500000000    | 11.335     | 11.335     | 0.203      | 7.084      | 3.449      | 0.008      |
    PASSED
1000000000   | 22.234     | 22.234     | 0.404      | 13.825     | 6.820      | 0.008      |
    PASSED

Fingerprints (must be identical in every mode for the same size):
  100000000    caesar=4e0e8be3e8bb67b6b0ddf7db719f618e  xor=786f581151c23bfb053cbcbc1ba75206
  500000000    caesar=5ffff614c5d69bec8ea5fd4632e91c0a  xor=e97fcf0870d164306293f38107bf1d58
  1000000000   caesar=660c19db5379e09d1d03129bd9b6d3ed  xor=605355aa741fe406fb38e3ad4b90ed62

# CSV: mode,workers,size,time,time_avg,caesar,xor,gen,overhead,check
CSV,local,4,100000000,2.3874,2.3874,0.0442,1.4754,0.7374,0.0091,PASSED
CSV,local,4,500000000,11.3353,11.3353,0.2029,7.0840,3.4491,0.0082,PASSED
CSV,local,4,1000000000,22.2339,22.2339,0.4043,13.8254,6.8200,0.0081,PASSED
```